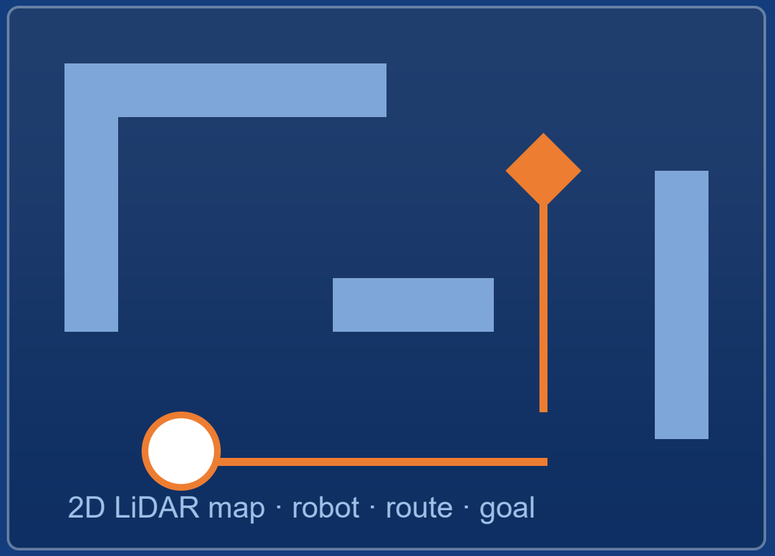
PROJECT BRIEFING

RB3302 Final Project

ROS 2 Mapping, Localization, and Autonomous Navigation

From sensor measurements to autonomous robot navigation



4 September 2026

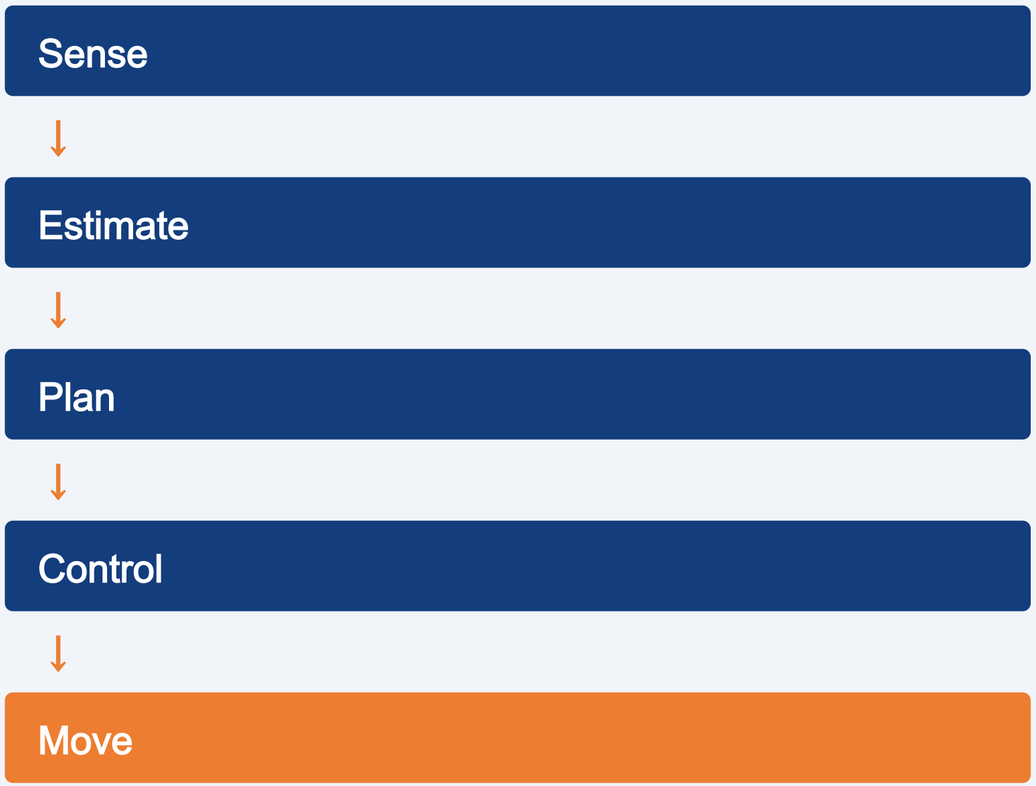
AY 25/26 Sem 1

NUS National University of Singapore | Advanced Robotics Centre

The Challenge

Develop a ROS 2 system that lets a mobile robot operate in an environment it has never seen before.

- Understand its surroundings
- Build a map
- Determine where it is
- Plan a route to a goal
- Navigate safely through the environment



OBJECTIVES

What You Will Build

By the end of the project, the robot should be able to:

01

Start and communicate with its sensors

02

Build an occupancy map

03

Save and reuse the map

04

Localise itself within the map

05

Plan and execute a collision-free route in sequence

PLATFORM

Hardware for the project

The robot and laptops are labelled based group numbers





Robot Platform

1 Laptop

laptop pw: rb3302

PLATFORM

Robot Platform



- Wave Rover differential-drive robot
- 2D RPLIDAR
- Yesense IMU
- Onboard ROS 2 computer
- Differential-drive mobile base

PLATFORM

ROS 2 Software Environment

Drivers

Robot base, LiDAR, IMU interfaces

RViz

Visualize sensors, frames, maps, plans

Mapping with Cartographer

Lidar-based mapping

On the robot: in a new terminal

```
ssh ubuntu@192.168.50.<GROUP NUMBER>
```

pw: rubikpi

REPOSITORY

Step 1 : Running the Drivers

robot_ws

Robot-side components — provided

hardware drivers
sensor interfaces
robot model
TF configuration

On the robot: in a new terminal

```
ros2 launch robot_init robot.launch.py
```

REPOSITORY

Step 2 : Running the teleoperation

robot_ws

Robot-side components — provided

hardware drivers
sensor interfaces
robot model
TF configuration

running

```
ros2 launch robot_init robot.launch.py
```

On the robot: in a new terminal

```
ros2 run waverover_base keyboard_teleop
```

REPOSITORY

Step 3 : Running the mapping algorithm

robot_ws

Robot-side components — provided

hardware drivers
sensor interfaces
robot model
TF configuration

running

```
ros2 launch robot_init robot.launch.py
```

running

```
ros2 run waverover_base keyboard_teleop
```

student_ws

Where your mapping and navigation work happens

- mapping_cartographer
- planning_astar
- planning_rpp

On the robot: in a new terminal

```
ros2 launch mapping_cartographer mapping.launch.py
```

REPOSITORY

Step 4 : Running the visualisation

robot_ws

Robot-side components — provided

hardware drivers
sensor interfaces
robot model
TF configuration

running

```
ros2 launch robot_init robot.launch.py
```

running

```
ros2 run waverover_base keyboard_teleop
```

student_ws

Where your mapping and navigation work happens

mapping_cartographer

planning_astar

planning_rpp

running

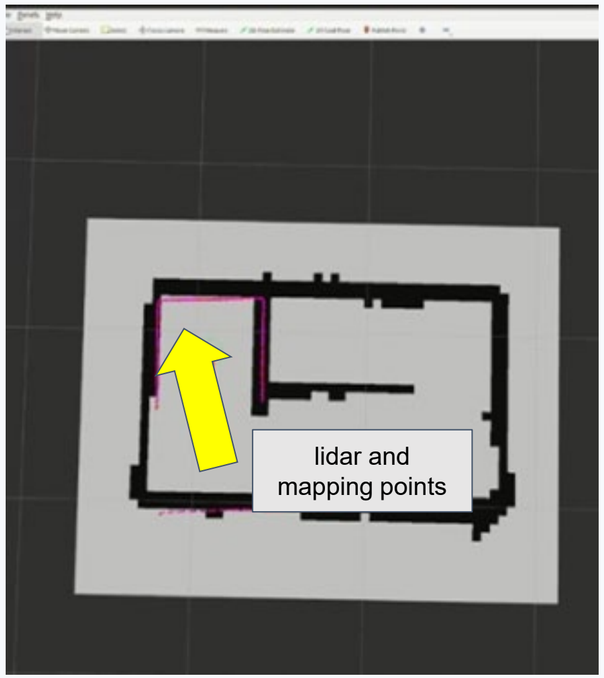
```
ros2 launch mapping_cartographer mapping.launch.py
```

On your laptop: in a new terminal

```
rviz
```

TASK 1

Mapping: with Cartographer and IMU



Task 2: Localisation

Once a map exists, how does the robot determine where it is?

AMCL

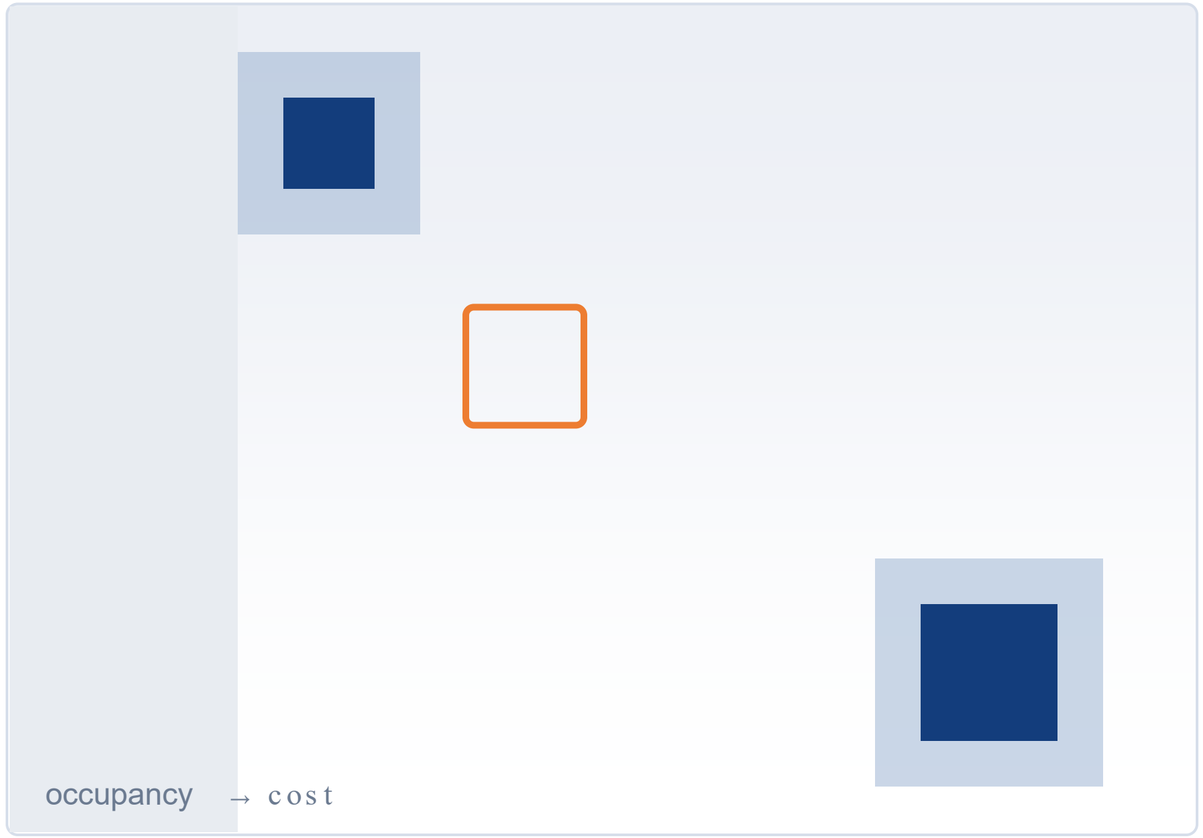
- Particle-based localisation — many pose hypotheses at once
- A saved occupancy map as the reference
- An initial pose to start from



particle cloud converging on the true pose

TASK 3

Costmaps and Safe Navigation



- Free space
- Obstacle
- Unknown space
- Inflation / safety region
- Robot footprint

Global costmap

The whole known map – used to plan a route

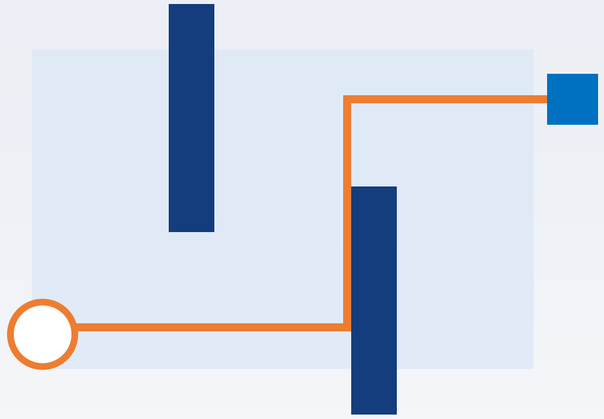
Local costmap

A small window around the robot – used to move safely

How much space should the planner leave between the robot and an obstacle? An incorrect footprint or inflation setting can lead to unsafe navigation.

TASK 3

Global Path Planning



start · search region · obstacles · goal · path

PURPOSE

Given the robot's current location, the goal and the map, find a feasible route through the environment.

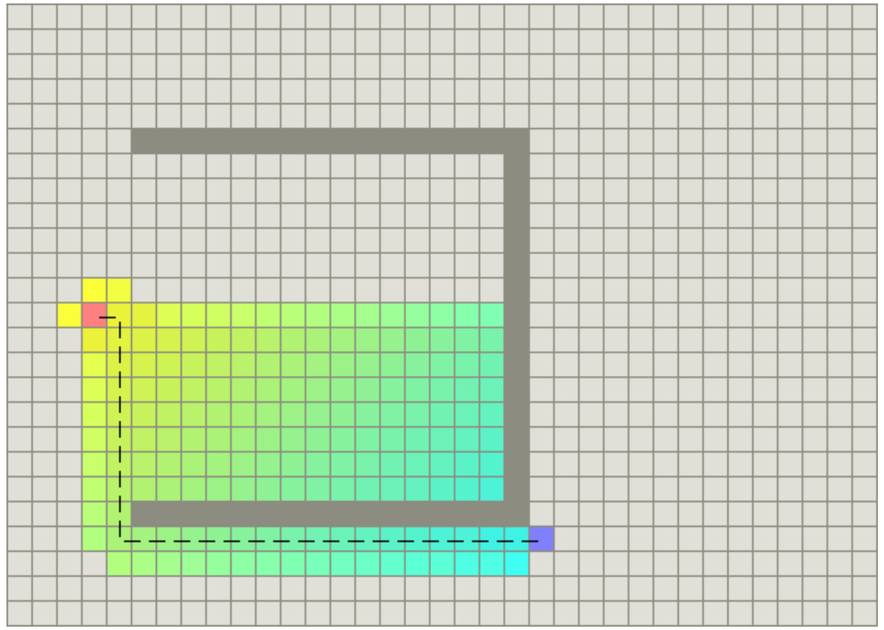
approach used in this project: A*

- What makes one path preferable to another?
- How are obstacles presented to the planner?
- When should the global path be recomputed?

TASK 3

Following the Path

```
ros2 launch planning_astar planning.launch.py
```



A*

GLOBAL PLANNER

Determines **where** the robot should go

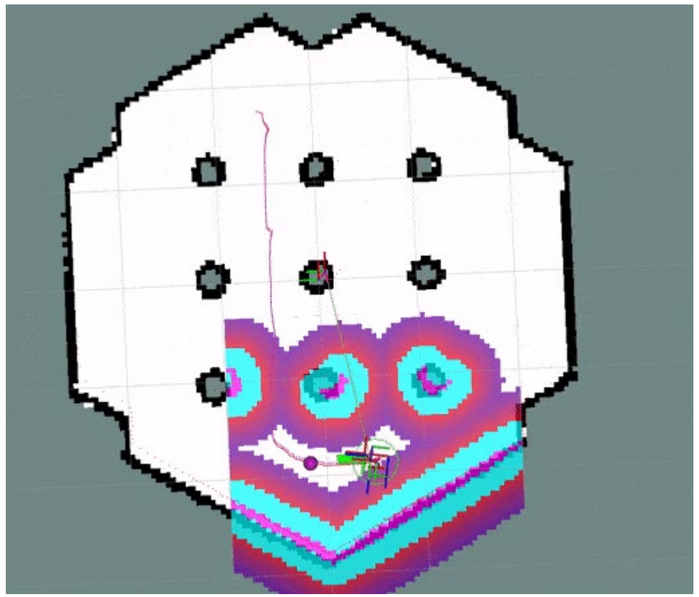
CONCEPTS TO INVESTIGATE

Lookahead	Curvature	Linear / angular velocity	Collision checking	Speed regulation

TASK 3

Following the Path

```
ros2 launch planning_rpp planning.launch.py
```



Regulated Pure Pursuit

LOCAL CONTROLLER

Determines **how** the robot moves along that path

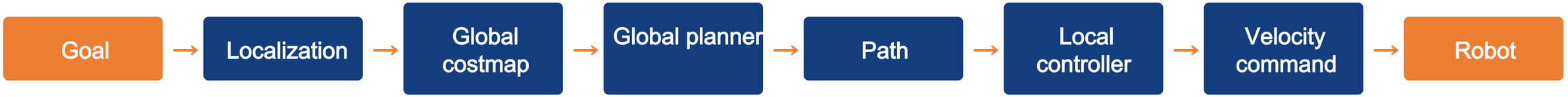
https://docs.ros.org/en/humble/p/nav2_regulated_pure_pursuit_controller/

CONCEPTS TO INVESTIGATE

Lookahead	Curvature	Linear / angular velocity	Collision checking	Speed regulation

INTEGRATION

Putting Everything Together



↶ Sensors keep updating the robot's understanding of the environment while it moves the loop never stops running.

TWO NAVIGATION PACKAGES

- planning_astar
- planning_rpp